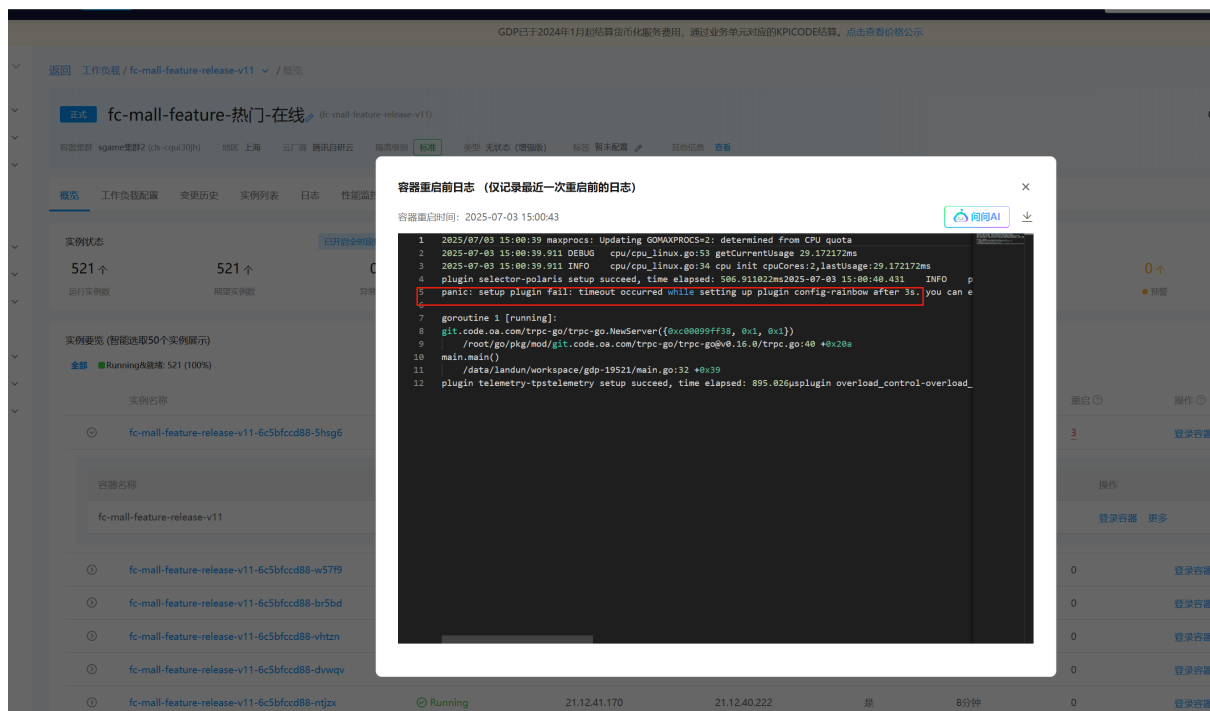


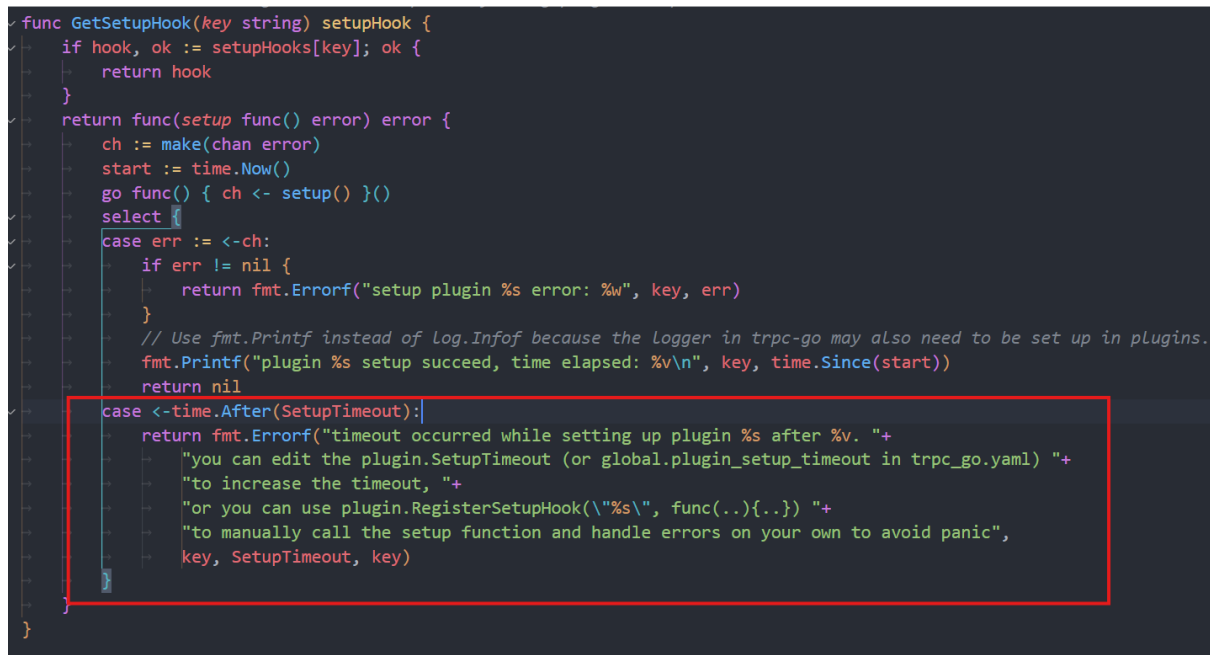
解决七彩石超时

1. 现象：

在晚高峰时期（特别是新赛季更新前几天），拉取七彩石配置出现超时现象，即初始化七彩石插件耗时超过3秒，发出告警，导致扩容后的容器实例要重启几次才能成功。



代码触发点：



超时是对于单个插件来说的，比如log-base超时3秒、config-rainbow超时3秒：

```
func (p *pluginInfo) setup() error {
    return GetSetupHook(p.key())(
        func() error {
            return p.factory.Setup(p.name, p.decoder)
        })
}
```

2. 可能原因：

1、自己猜测的可能原因：

- DNS解析超时：使用域名http://api.rainbow.oa.com:8080接入时，如果同一时间有大量其它服务同时进行DNS解析时，会导致DNS响应时间延长，从而使拉取七彩石配置超时；
- 网络拥堵：在晚高峰时，整体网络流量增大，可能导致容器实例与七彩石之间的网络连接变得拥堵，导致延迟增加，从而出现超时；
- 七彩石平台延迟：晚高峰时期七彩石平台自身面临高并发，响应时间变长，超出容器启动阶段的等待阈值。

以上原因都不太可能，因为现网每分钟大概1000w的请求，因此不太可能是并发导致的延迟变高，并且我们所超时的服务在告警时间段也没有被限流。

2、和七彩石那边沟通后得到可能的原因：

- 域名访问会有随机性，可能存在跨地访问，这个比较影响耗时；
- 再就是可能会有网络抖动，也会影响。

3. 复现过程：

1、本地并发拉取七彩石配置：

```
1 LOG_FILE="all_tasks.log" # 统一日志文件名
2 > "$LOG_FILE"          # 清空旧日志
3
4 # 完全并发执行（无并发数限制）
5 for i in {1..300}; do
6     {
7         ./server.sh >> "$LOG_FILE" 2>&1
8     } & # 后台执行任务
9 done
10
11 wait # 等待所有后台任务完成
12 echo "All tasks completed. Logs saved to $LOG_FILE"
```

```

2025/07/07 19:14:53 ip.go:144: [0:37m[info] 2081 ip: 21.6.68.166[0m
plugin config-rainbow setup succeed, time elapsed: 1.895609274s
plugin config-rainbow setup succeed, time elapsed: 1.179422135s
2025/07/07 19:14:53 ip.go:125: [0:36m[debug] set locap ip from server: 30.163.240.131:8080[0m
2025/07/07 19:14:53 ip.go:144: [0:37m[info] local ip: 21.6.68.166[0m
2025/07/07 19:14:53 ip.go:125: [0:36m[debug] set locap ip from server: 30.163.240.131:8080[0m
2025/07/07 19:14:53 ip.go:144: [0:37m[info] local ip: 21.6.68.166[0m
2025/07/07 19:14:53 ip.go:125: [0:36m[debug] set locap ip from server: 30.163.240.131:8080[0m
2025/07/07 19:14:53 ip.go:144: [0:37m[info] local ip: 21.6.68.166[0m
plugin config-rainbow setup succeed, time elapsed: 1.393980344s
[0:36m[debug] set locap ip from server: 30.163.240.131:8080[0m
2025/07/07 19:14:53 ip.go:144: [0:37m[info] local ip: 21.6.68.166[0m
plugin config-rainbow setup succeed, time elapsed: 659.889687ms
and c2nc2c0m1ond2nc0m1ond2c0gCAPC00atS: server/server
setup plugin fail: timeout occurred while setting up plugin config-rainbow after 3s. you can edit the plugin.SetupTimeout (or global.plugin_setup_timeout in trpc-go.yaml) to increase the tim
goroutine 10000 [0:37m[info] ...
2025/07/07 19:14:53 ip.go:125: [0:36m[debug] set locap ip from server: 30.163.240.131:8080[0m
2025/07/07 19:14:53 ip.go:125: [0:36m[debug] set locap ip from server: 30.163.240.131:8080[0m
2025/07/07 19:14:53 ip.go:144: [0:37m[info] local ip: 21.6.68.166[0m
[running]:
git.code.oa.com/trpc-go/trpc-go.NewServer({0x0, 0x0, 0x0})
/root/pkg/mod/git.code.oa.com/trpc-go/trpc-go@v0.20.0/trpc-go:51 +0x245
main.main()

```

此复现结果无意义，因为并发不是导致超时的原因，本机复现出现这种情况是由于自身并发处理能力不足导致的延迟增大。

2、本地串行拉取七彩石配置耗时：

```

1 LOG_FILE="all_tasks.log" # 统一日志文件名
2 > "$LOG_FILE" # 清空旧日志
3 for i in {1..300}; do
4 {
5 ./server.sh >> "$LOG_FILE" 2>&1
6 } & # 后台执行任务
7
8 sleep 3 # 关键修改：等待3秒再启动下一个任务
9 done

```

```

C:\Users\...> .\server.sh
SetupPlugins cost: 8.761858ms [0:37m[info] ...
and c2nc2c0m1ond2nc0m1ond2c0gCAPC00atS: server/server
[0:37m[info] ...
2025/07/07 19:29:34 ip.go:125: [0:36m[debug] set locap ip from server: 30.163.240.131:8080[0m
2025/07/07 19:29:34 ip.go:144: [0:37m[info] local ip: 21.6.68.166[0m
plugin config-rainbow setup succeed, time elapsed: 13.172851ms
2025-07-07 19:29:34.128 DEBUG maxprocs/maxprocs.go:48 maxprocs: Updating GOMAXPROCS=16: determined from CPU quota
2025-07-07 19:29:34.129 DEBUG debuglog@v0.1.13/log.go:229 client request:/trpc.redis.sliksong_pluginintimeout.service/GET, cost:355.121AIs, to:127.0.0.1:6379
SetupTimeout: 6
2025-07-07 19:29:34.130 INFO tnet/server_transport.go:100 service: hello.Greeter is using tnet tcp transport, current number of pollers: 1
2025-07-07 19:29:34.130 INFO server/service.go:207 process: 202108, trpc service: hello.Greeter launch success, tcp: 21.6.68.166:8080, serving ...
2025-07-07 19:29:34:08:00 [0:34m[info] tnet@v0.1.0/tcpserver.go:88 tnet tcp service started, current number of pollers: 1, use tnet.SetNumPollers to change it

SetupPlugins cost: 9.161279ms [0:37m[info] ...
and c2nc2c0m1ond2nc0m1ond2c0gCAPC00atS: server/server
[0:37m[info] ...
2025/07/07 19:29:37 ip.go:125: [0:36m[debug] set locap ip from server: 30.163.240.131:8080[0m
2025/07/07 19:29:37 ip.go:144: [0:37m[info] local ip: 21.6.68.166[0m
plugin config-rainbow setup succeed, time elapsed: 12.300453ms
2025-07-07 19:29:37.129 DEBUG maxprocs/maxprocs.go:48 maxprocs: Updating GOMAXPROCS=16: determined from CPU quota
2025-07-07 19:29:37.129 DEBUG debuglog@v0.1.13/log.go:229 client request:/trpc.redis.sliksong_pluginintimeout.service/GET, cost:431.102AIs, to:127.0.0.1:6379
SetupTimeout: 6
2025-07-07 19:29:37.130 INFO tnet/server_transport.go:100 service: hello.Greeter is using tnet tcp transport, current number of pollers: 1
2025-07-07 19:29:37:08:00 [0:34m[info] tnet@v0.1.0/tcpserver.go:88 tnet tcp service started, current number of pollers: 1, use tnet.SetNumPollers to change it
2025-07-07 19:29:37.131 INFO server/service.go:207 process: 202441, trpc service: hello.Greeter launch success, tcp: 21.6.68.166:8080, serving ...

```

此开发环境中模拟的串行拉取得到的插件初始化时间大多都是毫秒级，但是此复现结果也意义不大，因为这种情况也不能完全复现出idc环境中真实的超时的场景。

不管是在本地的串行拉取七彩石还是并行拉取七彩石都不能对真实情况实现准确复现，一个是开发机环境，一个是现网环境，很难实现场景统一。本机复现结果参考价值不大，只能在横向对比中有可信之处。

4. 目前解决方案：

- 将域名从http://api.rainbow.oa.com:8080改成http://api.rainbow.woa.com:8080，因为oa的域名不太稳定，可以换成woa；

- 换成北极星接入polaris://65026305:65536，因为域名访问会有随机性，可能存在跨地访问，这个比较影响耗时；而北极星是就近访问，因此会有改善；
- 将代码中使用的trpc-config-rainbow版本升级，建议升级到0.4.4，新版本容灾能力更强，这个改进理论上与拉取配置超时无关；
- 兜底方法，直接改trpc-config-rainbow包中设置的插件超时时间plugin.SetupTimeout，如：
 - 手动改动每个服务的超时时间：

```
1 // 方法一：代码修改
2 import (
3     "git.code.oa.com/trpc-go/trpc-go/plugin"
4     "time"
5 )
6
7 const pluginSetupTimeout = time.Second * 10
8
9 // 修改每个插件的超时时间
10 func initTRPCConfig() {
11     plugin.SetupTimeout = pluginSetupTimeout
12 }
13 // 注意：上述方法设置超时时间需要在NewServer之前，因为初始化插件是在NewServer
    中进行的
14
15 // 方法2：yaml配置文件直接修改（这样设置的所有插件的超时时间一样）
16 global:
17     plugin_setup_timeout: ***
```

- 超时时间写入redis，此种方案用于两种情况：
 - 对于一个服务，它的超时时间动态变化幅度较大；
 - 对于一个服务，事先不清楚它的超时时间，这样可以在重启的过程中自动一次一次的更新redis中的超时时间值：

```
1  1.
2  // proxies/redis_client.go
3  var SilkSongPluginTimeoutRedis =
    redis.NewClientProxy("trpc.redis.sliksong_plugintimeout.service")
4
5
6  2.
7  // setplugintimeout/timeoutprocess.go
8  const maxTimeout = time.Minute
9
10 func Init() {
11     // 从redis中获取超时时间
12     SetupTimeout, err :=
        redis.String(proxies.SilkSongPluginTimeoutRedis.Do(context.Background(), "GET",
            "timeout"))
13     fmt.Println(SetupTimeout)
14     if err == nil && SetupTimeout != "" { // 键存在且无错误，否则超时时间是默认值
15         fmt.Println("SetupTimeout:", SetupTimeout)
16         val, _ := strconv.Atoi(SetupTimeout)
17         plugin.SetupTimeout = time.Duration(val) * time.Second
18     }
19 }
20
21 // 超时一定时间后，使用降级策略处理错误，pluginKey为插件名称
22 func Process(pluginKey string) {
23     plugin.RegisterSetupHook(pluginKey, func(setup func() error) error {
24         ch := make(chan error)
25         go func() { ch <- setup() })
26         select {
27         case err := <-ch:
28             return err // 正常完成，返回错误或nil
29         case <-time.After(plugin.SetupTimeout):
30             timeoutCallback() // 调用钩子函数
```

方法2的本地测试结果：

1. 一开始redis中没有超时时间对应的键值对：

```
[root@drewjjli-1t70osepea workspace]# redis-cli
127.0.0.1:6379> keys *
(empty list or set)
127.0.0.1:6379> 
```

2. 当拉取七彩石配置超时，此时redis中就会自动存入对应的超时时间，其值等于默认超时时间的2倍=6秒：

```
panic: setup plugin fail: timeout occurred while setting up plugin config-rainbow after 3s. you can edit the plugin.SetupTimeout (or global.plugin_setup_timeout in trpc-go.yaml) to increase

goroutine 1 [running]:
git.code.oa.com/trpc-go/trpc-go.NewServer({0x0, 0x0, 0x0})
/root/go/pkg/mod/git.code.oa.com/trpc-go@v0.20.0/trpc-go:51 +0x245
main.main()
/dive/workspace/test/server/main.go:18 +0x16

127.0.0.1:6379> keys *
1) "timeout"
127.0.0.1:6379> get timeout
"6"
127.0.0.1:6379> 
```

需要注意：plugin.SetupTimeout直接控制的超时时间是每个插件共有的，如果只想对某个插件的超时时间控制就可以在yaml配置文件中添加对应插件的配置信息即可，如七彩石：

```
type Config struct {
    ClientProviderMode string `yaml:"client_provider_mode" // 取代或融合 client config`
    OpenSign           bool  `yaml:"enable_sign"`
    Addr               string `yaml:"address" // Deprecated, 推荐业务用上面的接入点，但是特殊`
    Name               string `yaml:"name"`
    AppID              string `yaml:"appid"`
    Group              string `yaml:"group"`
    LocalIP            string `yaml:"local_ip"`
    IsRemote           bool  `yaml:"is_remote"`
    EnvName            string `yaml:"env_name"`
    UseGlobalEnvName   *bool `yaml:"use_global_env_name"`
    Type               string `yaml:"type" // 设置为 group 时，可以配置下面的 GroupType`
    // 当 Type 配置为 group 时，GroupType 才生效，生效时默认为 kv。目前只支持 kv 和 file。
    // trpc-config-rainbow 版本 >= v0.5.2 时，才支持 GroupType 字段。
    GroupType          string `yaml:"group_type"`
    Uin                 string `yaml:"uin"`
    FileCachePath       string `yaml:"file_cache"`
    IsUsingFileCache    *bool `yaml:"is_using_file_cache" // default true`
    UserID              string `yaml:"user_id"`
    UserKey             string `yaml:"user_key"`
    HmacWay             string `yaml:"hmac_way"`
    Timeout             int   `yaml:"timeout" |`
    ClientInfo          map[string]string `yaml:"client_info"`
    DownloadFileAddrType string `yaml:"download_file_addr_type"`
    LogLevel            string `yaml:"log_level"`
    LogName             string `yaml:"log_name"`
    SkipInitialGetOnWatch bool  `yaml:"skip_initial_get_on_watch"`
}
```

最终结论：

1. 可以将七彩石的接入方式的域名从oa改到woa或者改为北极星接入，建议改为北极星方式，耗时更短；
2. 实在不行就走修改超时时间进行兜底，这里也有两种方式：一种是简单的直接修改服务的超时时间（首选方案，超时时间不会大幅度动态变化）；还有一种就是把每个服务调整后的超时时间存入redis，此类方法是用于应对超时时间动态变化和不知道具体超时时间值的情况。